

programming python

基本的なpythonの文法を学びなおす

```
score = 85
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "C"
print(f"점수: {score}, 학점: {grade}")
```

If文に関して、Javaと比べるとifとelseは同じだが、else ifの代わりにelifを使う。あと、printする時に{}の中に変数を埋め込んで出力したい時は、fをつける

```
abc = ["a", "b", "c"]
for alph in abc:
    print(alph)

for i in range(3):
    print(i, end=" ")
```

```
a
b
c
0 1 2
```

For文はIterableの要素を順番に取得することができる。
そして、rangeを使って取得する範囲を指定することができる。

```
count = 0
while count < 2:
    print(f"count: {count}")
    count += 1
```

```
count: 0
count: 1
```

While文はFalseになったら反復が止まる仕組みなので
このように範囲を指定することができる

```
def add_numbers(x, y):
    return x + y

result = add_numbers(5, 7)
print(f"result : {result}")
```

```
result : 12
```

関数はこのようにdefを付ける

```
my_list = [1, 2, 3]
my_list.append(4)
print(f"list : {my_list}")
my_list.pop()
print(f"list : {my_list}")
my_list.pop(1)
print(f"list : {my_list}")
```

```
list : [1, 2, 3, 4]
list : [1, 2, 3]
list : [1, 3]
```

Listは複数のデータを順番に保存することができるIterableオブジェクトである
このようにappend, popを使ってListの追加削除ができる

```
my_dict = {"name": "yun", "age": 28}
print(f"dict: {my_dict['name']}")
```

```
dict: yun
```

KeyとValueのペアでデータを保存するiterableなオブジェクトである
Keyは重複できず、変更不可能なデータ型のみ使用できるValueは重複も修正もできる

```
my_tuple = (1, 2, 3)
my_set = {1, 2, 2, 3}
```

tupleは作成したら、内部の要素を変更できないiterableなオブジェクトである
setは順序がなく、重複する値もできないiterableなオブジェクトである

```
class Player:
    def __init__(self, name, team):
        self.name = name
        self.xp = 1500
        self.team = team

    def introduce(self):
        print(f"hi my name is {self.name}")

nico = Player("nico", "LA")
jack = Player("jack", "NewYork")

nico.introduce()
jack.introduce()
```

クラスはJavaと比べると引数にselfを付けてクラスの変数に接近することが違う

Pythonを使ってクローリング

requests, BeautifulSoupを使用して、urlに接近、情報を取り出す

requests 2.34.2

Python HTTP for Humans.

`pip install requests`

[문서](#) [파일 다운로드](#) [릴리즈 이력](#)

Requests

pypi v2.34.2

python 3.10 | 3.11 | 3.12 | 3.13 | 3.14 | 3.15

downloads/month 2G

contributors 401

docs passing

Requests is a simple, yet elegant, HTTP library.

```
>>> import requests
>>> r = requests.get('https://httpbin.org/basic-auth/user/pass', auth=('user', 'pass'))
>>> r.status code
```

beautifulsoup4 4.15.0

Screen-scraping library

`pip install beautifulsoup4`

[문서](#) [파일 다운로드](#) [릴리즈 이력](#)

Beautiful Soup is a library that makes it easy to scrape information from web pages. It sits atop an HTML or XML parser, providing Pythonic idioms for iterating, searching, and modifying the parse tree.

Quick start

```
>>> from bs4 import BeautifulSoup
>>> soup = BeautifulSoup("<p>Some<b>bad<i>HTML")
>>> print(soup.prettify())
```

```

import requests
from bs4 import BeautifulSoup

url = "https://weworkremotely.com/categories/remote-full-stack-programming-jobs"

response = requests.get(url)

print(response.content)

soup = BeautifulSoup(response.content,
                        "html.parser")

jobs = soup.find("div", id="search-results").find("li")

print(jobs)

```

```

<a class="listing-link--unlocked" href="/remote-jobs/toptal-mechanical-engineer">
  <div class="new-listing paid-logo">
    <div class="new-listing__header">
      <h3 class="new-listing__header__title">
        <span class="new-listing__header__title__text">Mechanical
    </div>
    <p class="new-listing__company-name">
      " Toptal "

```

BeautifulSoupのfindメソッドを使ってid="search-results"のに接近

```

for job in jobs:
    title = job.find("span", class_="new-listing_header_title_text")
    company_name = job.find("p", class_="new-listing_company-name")
    print(title, "-----", company_name)

```

```

<span class="new-listing_header_title_text">Technical Leader - Latin America</span> ----- <p class="new-listing_company-name">Azumo </p>
<span class="new-listing_header_title_text">Senior Staff Data Engineer - Platform Data and Analytics</span> ----- <p class="new-listing_company-r
">Faire </p>
<span class="new-listing_header_title_text">Data Analyst, Reporting Partnerships</span> ----- <p class="new-listing_company-name">Doximity </p>

```

の中にあるtitle, company_nameの情報を取り出す

```
for job in jobs:
    title = job.find("span", class_="new-listing_header_title_text").get_text()
    company_name = job.find("p", class_="new-listing_company-name").get_text()
    print(title, "-----", company_name)
```

```
title = job.find("span", class_="new-listing_header_title_text").get_text()
AttributeError: 'NoneType' object has no attribute 'get text'
```

textだけを出しようとしてget_text()を使ったが、調べてみたら値のない部分もあってエラーが発生した

```
for job in jobs:

    title_element = job.find("span", class_="new-listing__header__title__text")
    if title_element:
        title = title_element.get_text()
    else:
        title="x"

    company_name_element = job.find("p", class_="new-listing__company-name")
    if company_name_element:
        company_name = company_name_element.get_text()
    else:
        company_name = "x"

    print(title, "-----", company_name)
```

値のない部分If文を使って"x"に処理

```
x ----- Remote Talent Cloud
Mechanical Engineer ----- Toptal
Staff Software Engineer ----- Samsara
Senior Independent AI Engineer / Architect ----- A.Team
x ----- Metana.io
Senior Independent Software Developer ($90-$170/hr) ----- A.Team
Senior Shopify Developer (Remote + Flexible) ----- Storetasker
Software Development Engineer II Full Stack - Business (Remote @ Colombia) ----- OfferUp
Senior Cloud Engineer (Hybrid @ Bellevue, WA or Remote @ Florida) ----- OfferUp
Director, GTM Finance ----- Customer.io
Medical Licensing Specialist (Contract) ----- Galileo
Senior Product Manager, Consumer Payments ----- Toast
Product Content Specialist ----- Traction
Senior Product Designer - ShareGate Migrate ----- Sharegate Migrate Chezworkleap
Manager/ Sr. Manager - Data Product Manager ----- Tiger Analytics Inc.
Senior Product Designer - ShareGate Migrate ----- Workleap
Senior Platform Product Manager (Contract Position) ----- Pindrop
Senior Technical Product Manager ----- Simprints
Staff Product Manager ----- Brightwheel
Technical Product Marketer - KØrdent AI ----- Mirantis
```

結果、textを取り出すことができる